

ngspice commands — a worked reference

Every command in this ngspice-46 + OpenVAF-r build, with netlists and plots

Ngspice + OpenVAF Enhancements project

September 2026

Contents

ngspice commands — a worked reference	1
1. How commands are run	1
2. Analyses	2
3. RF and periodic steady-state	8
4. Signal processing and measurement	11
5. Vectors, variables and expressions	12
6. Plotting and data export	13
7. Circuit and device management	14
8. Compiling and loading models	14
9. Statistics, yield and optimization	15
10. Post-layout and reliability	16
11. Interactive debugging and control	16
12. Control-flow scripting (.control)	16
13. Help and utility commands	17
14. Every command help all lists	17

ngspice commands — a worked reference

This document walks through the commands available in this ngspice build — the stock nutmeg/spice command set plus every command added by the [enhancement project](#) (the RF steady-state suite, the optimizer, Monte-Carlo/yield, post-layout reliability, pyplot, pre_osdi/pre_snp, eye, stb, ...). The core analyses are shown with complete, copy-pasteable netlists and plots produced by actually running them; the many smaller utility commands are collected into reference tables at the end, and the last section lists every command this build's **help all** prints, with its one-line description, generated from a real run.

Every plot in this document was generated by [make_commands_figs.py](#) from the exact netlists shown.

1. How commands are run

An ngspice input file (a *deck* or *netlist*) has this shape — the first line is always the title and is ignored as a component:

```
* my circuit title          <- title line (ignored)
V1 in 0 dc 1                <- components
R1 in out 1k
C1 out 0 1u
```

```
.tran 10u 5m          <- a "dot-card" analysis
.control              <- an interactive command block
run
plot v(out)
.endc
.end
```

There are four ways a command runs:

Context	How	Example
Batch	ngspice -b deck.cir runs every dot-card analysis and any .control block, then exits	ngspice -b amp.cir
Interactive	at the ngspice N -> prompt	ngspice -> tran 1u 1m run then print v(out)
.control block	commands between .control and .endc in the deck	.ac dec 10 1 1meg
Dot-card	.tran/.ac/... in the deck body, executed by run	

The interactive analysis commands (tran, ac, dc, op, ...) take the same arguments as their dot-card spelling and run immediately. run executes the analysis named by the deck's dot-cards. Results are stored as named vectors inside plots (print, plot, and expressions read them back).

2. Analyses

op — DC operating point

Solves the quiescent (DC) bias point: every node voltage and source current with all capacitors open and inductors shorted.

```
* operating point of a resistive divider
V1 in 0 dc 5
R1 in out 2k
R2 out 0 3k
.control
op
print v(out) i(v1)
.endc
.end
```

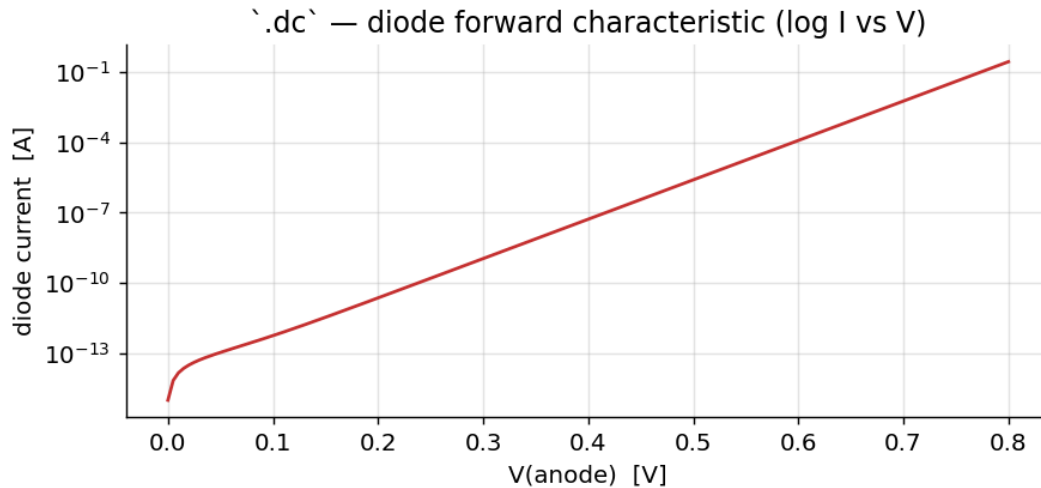
v(out) = 3.0 V, i(v1) = -1 mA. op is also the bias point every AC, noise, tf, pz, and small-signal analysis linearizes around.

dc — DC sweep (and nested sweep)

dc <src> <start> <stop> <step> [<src2> <start2> <stop2> <step2>] sweeps a source (and optionally a second, giving a family of curves). The classic uses are a diode I-V and a transistor's output characteristics.

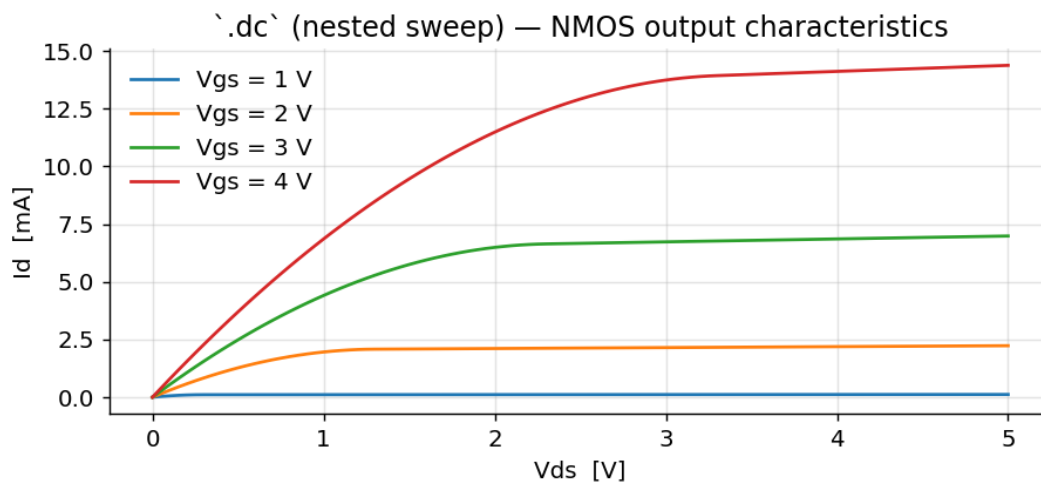
```
* diode forward I-V
Vd a 0 dc 0
D1 a 0 DMOD
.model DMOD D(is=1e-14 n=1)
```

```
.dc Vd 0 0.8 0.005
.control
run
plot abs(i(vd)) ylog
.endc
.end
```



A nested sweep produces the textbook MOSFET output family — the inner source V_{ds} sweeps for each value of the outer source V_{gs} :

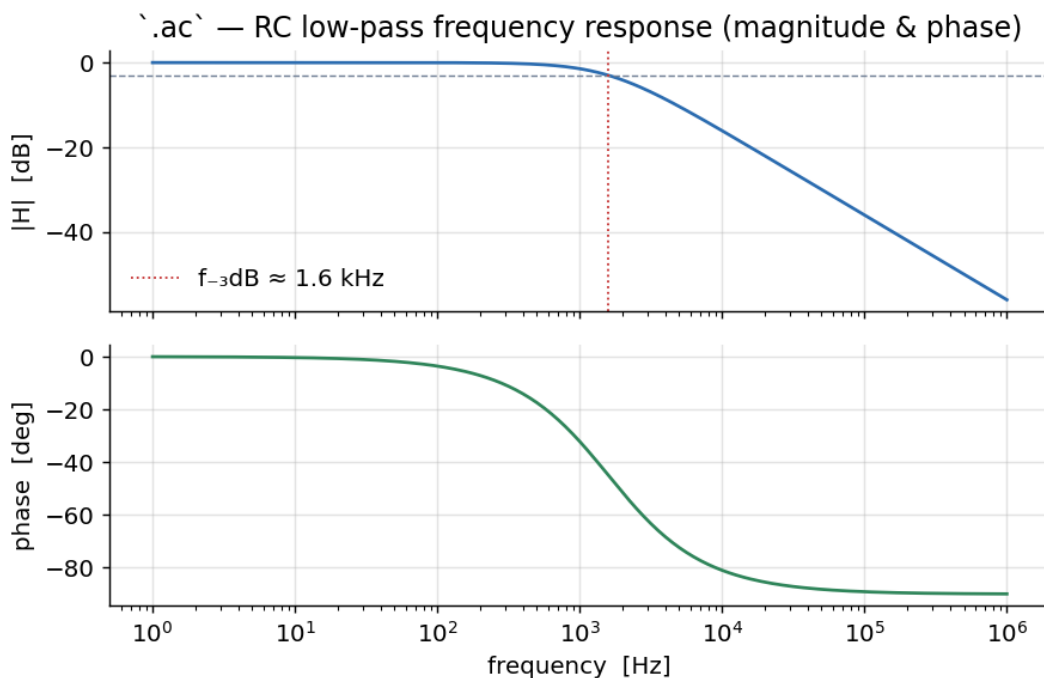
```
* NMOS output characteristics (nested .dc)
Vds d 0 dc 0
Vgs g 0 dc 0
M1 d g 0 0 NM W=20u L=1u
.model NM NMOS(level=1 vto=0.7 kp=120u lambda=0.02)
.dc Vds 0 5 0.05 Vgs 1 4 1
.control
run
plot -i(vds)
.endc
.end
```



ac — small-signal frequency response

`ac <lin|dec|oct> <N> <fstart> <fstopt>` linearizes at the operating point and sweeps frequency. Drive the input with an `ac` magnitude and read `vdb()` (magnitude in dB) and `vp()` (phase).

```
* RC low-pass Bode
Vin in 0 dc 0 ac 1
R1 in out 1k
C1 out 0 100n
.ac dec 40 1 1meg
.control
run
plot vdb(out)
plot vp(out)
.endc
.end
```



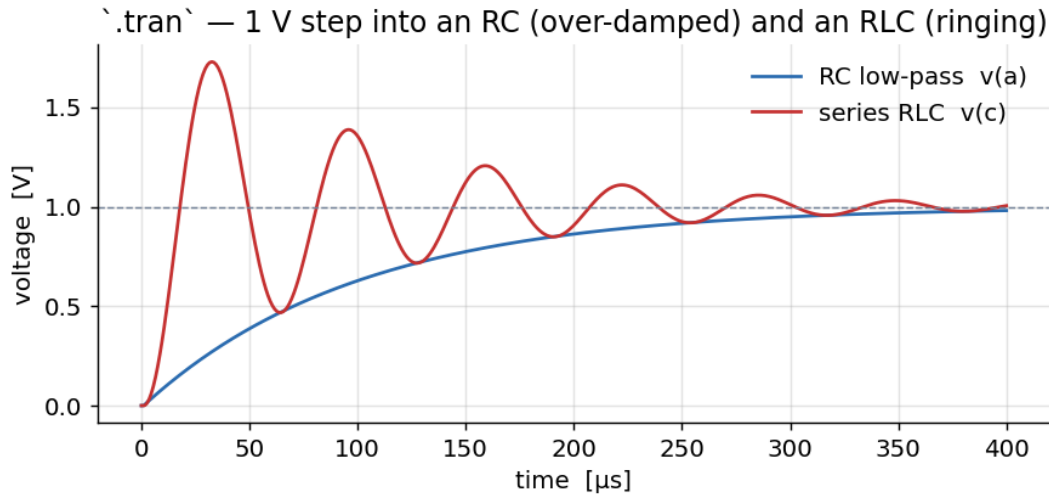
The -3 dB corner sits at $1/(2\pi RC) \approx 1.6$ kHz; the phase rolls from 0° to -90° .

tran — transient (time-domain)

`tran <tstep> <tstop> [tstart [tmax]] [uic]` integrates the circuit in time.

```
* 1 V step into an RC (over-damped) and a series RLC (ringing)
Vin in 0 pulse(0 1 1u 1n 1n 1 2)
R1 in a 1k
C1 a 0 100n
L1 in b 1m
Rs b c 20
C2 c 0 100n
.tran 1u 400u
.control
run
plot v(a) v(c)
.endc
```

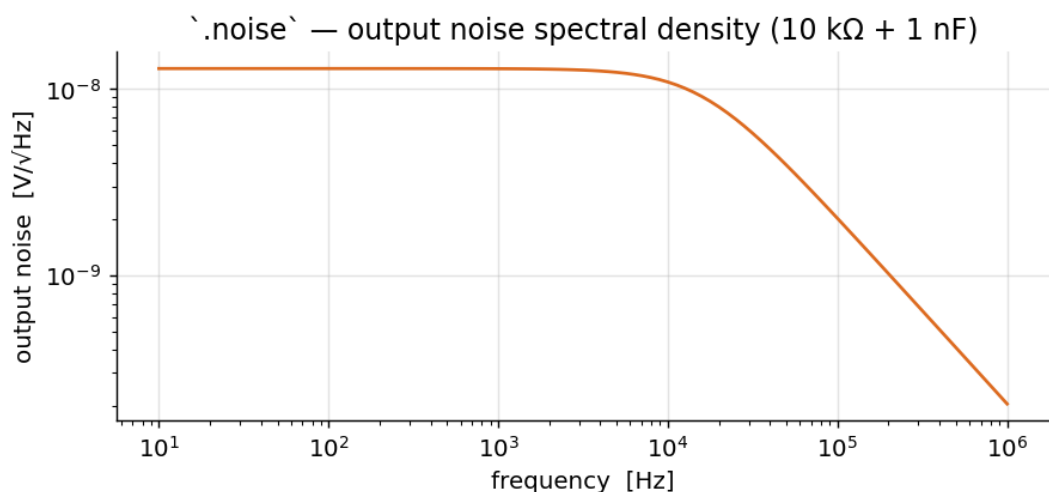
.end



noise — noise analysis

`noise <output> <input-source> <lin|dec|oct> <N> <fstart> <fstop>` computes the output- and input-referred noise spectral density (and integrated noise). The spectrum vectors live in the `noise1` plot; the integrated totals in `noise2`.

```
* output noise of a 10k + 1n RC
Vin in 0 dc 0 ac 1
R1 in out 10k
C1 out 0 1n
.noise v(out) Vin dec 20 10 1meg
.control
run
setplot noise1
plot onoise_spectrum
.endc
.end
```



tf — DC transfer function

`tf <output> <input-source>` reports the small-signal DC gain plus the input and output impedance. `<output>` is `v(node)`, `v(node1,node2)`, or `i(vsource)`.

* transfer function of a voltage divider

Vin in 0 dc 1

R1 in out 1k

R2 out 0 1k

.control

tf v(out) Vin

.endc

.end

Reports transfer_function = 0.5, output_impedance_at_v(out) = 500 Ω , vin#input_impedance = 2 k Ω .

pz — pole/zero

pz <n1> <n2> <n3> <n4> <cur|vol> <pz|pol|zer> finds the poles and zeros of the transfer function between the input node pair (n1,n2) and output pair (n3,n4).

* poles/zeros of a series RLC (v(out)/vin)

Vin in 0 dc 0 ac 1

L1 in m 1m

R1 m out 60

C1 out 0 100n

.pz in 0 out 0 vol pz

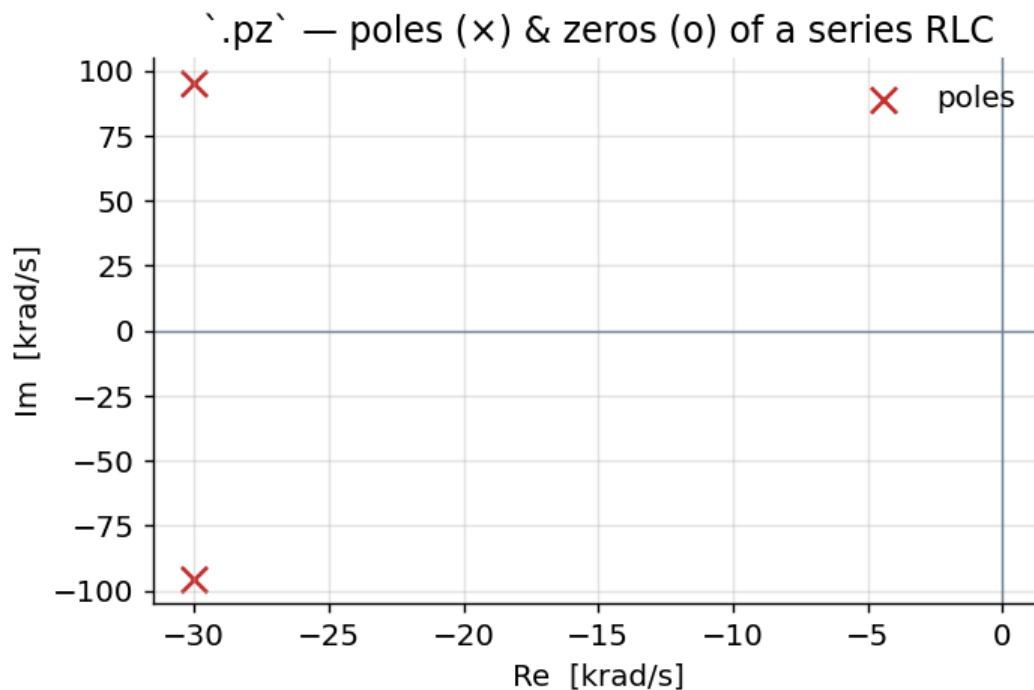
.control

run

print all

.endc

.end



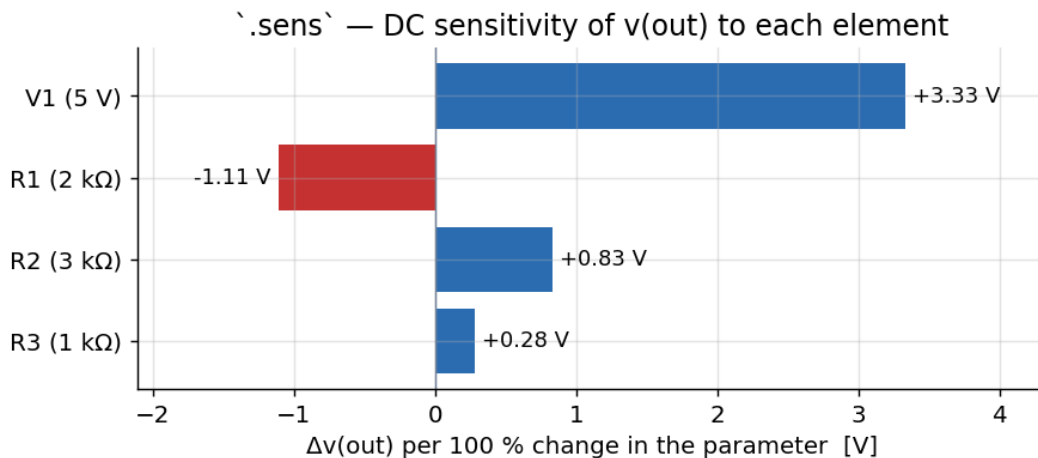
The complex-conjugate pole pair sits at -30 ± 95 krad/s — a resonance near $\omega_0 = 1/\sqrt{LC} = 100$ krad/s with the loss set by R.

sens — sensitivity

sens <output> computes the DC sensitivity of an output to every circuit parameter; append **ac** <lin|dec|oct> <N> <fstart> <fstopt> for the small-signal AC sensitivity vs frequency. The result lands in a **sens1** plot, one vector per device (its value sensitivity) and per model parameter (<dev>: <param>).

```
* DC sensitivity of v(out) to every element
V1 in 0 dc 5
R1 in out 2k
R2 out mid 3k
R3 mid 0 1k
.sens v(out)
.control
run
print v1 r1 r2 r3      ; each is  $\partial v(\text{out})/\partial \text{param}$ 
.endc
.end
```

Scaling each raw sensitivity by the parameter's nominal value gives a comparable “ $\Delta v(\text{out})$ per 100 % change” — the clearest way to rank which element matters most:



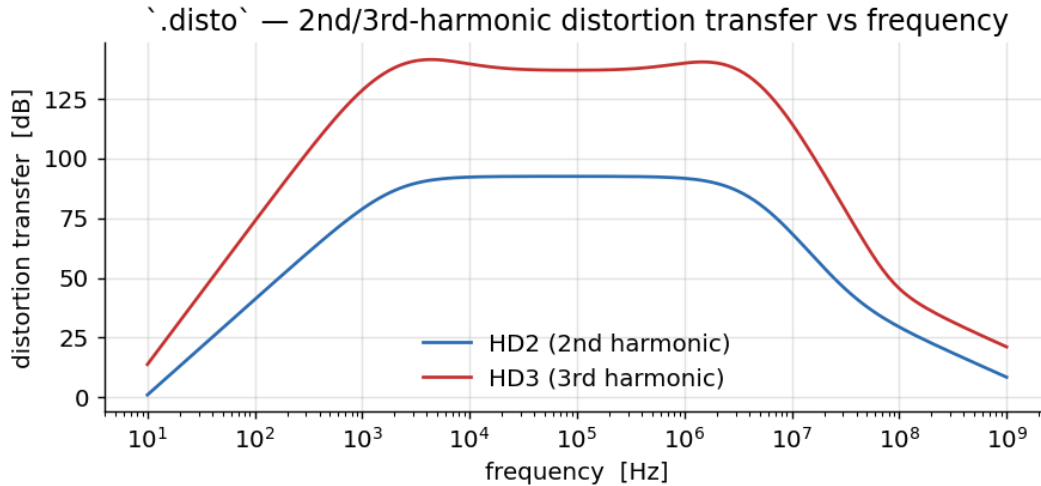
The output tracks the 5 V supply most strongly, and increasing the series R_1 *lowers* it (negative sensitivity), as expected for a divider.

disto — small-signal distortion

disto <lin|dec|oct> <N> <fstart> <fstopt> [f2overf1] sweeps a small-signal harmonic (and, with a second tone, intermodulation) distortion analysis. Mark the driving source with **distof1** (and **distof2** for two-tone). The 2nd- and 3rd-harmonic products land in the **disto1** / **disto2** plots.

```
* small-signal distortion of a common-emitter BJT amp
Vcc cc 0 dc 5
Vin in 0 dc 0.75 ac 1 distof1 1
Rb cc b 220k
Rc cc c 2.2k
Cin in b 1u
Cout c 0 20p
Q1 c b 0 QMOD
.model QMOD NPN(is=1e-15 bf=150 vaf=80 cjc=3p tf=0.3n)
.disto dec 20 10 1g
```

```
.control
run
setplot disto1
plot vdb(c)          ; HD2 vs frequency (disto2 holds HD3)
.endc
.end
```



The distortion (here in dB relative to the unit `distof1` drive) is flat across the amplifier's passband and rolls off with the input coupling capacitor at low frequency and the transistor's bandwidth at high frequency.

fourier

`fourier <fund_freq> <vec>` prints the DC term, the first harmonics, and the THD of a time-domain waveform — run a `.tran` first, then `fourier 10k v(out)`. (The `spec/fft` commands in §4 give the full spectrum as a plot.)

Event-driven (XSPICE) and control-flow commands

These are stock ngspice commands that this reference had not previously listed.

Command	What it does
<code>esave</code>	select event-driven (digital/XSPICE) nodes to record
<code>eprint</code>	print the event-driven node values recorded by <code>esave</code>
<code>edisplay</code>	list the event-driven nodes available in the current circuit
<code>eprvcd</code>	dump event-driven node activity as a VCD file for a waveform viewer
<code>foreach</code>	control-flow: iterate a command block over a word list (<code>foreach x 1 2 3 ... end</code>)
<code>repeat</code>	control-flow: run a command block a fixed number of times

3. RF and periodic steady-state

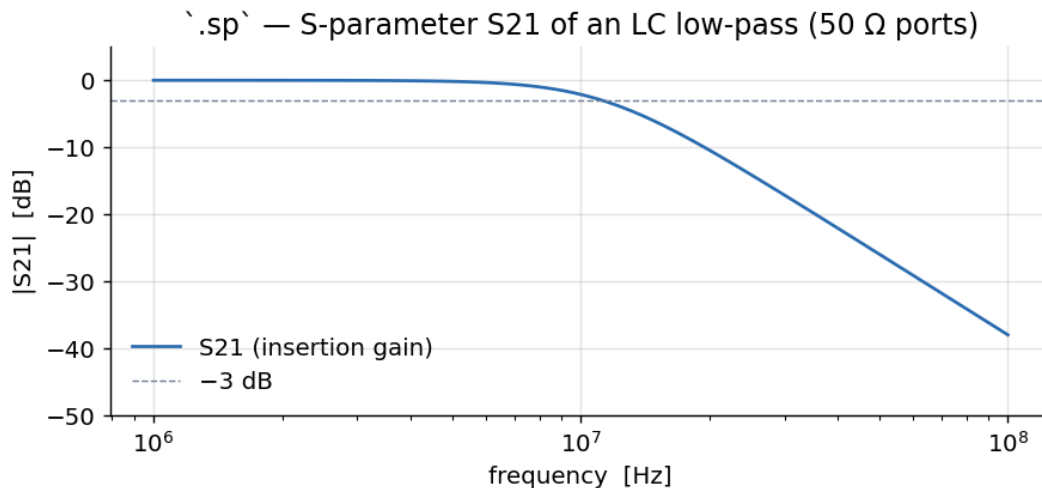
The project adds a full RF suite on top of the analyses above. Most run as their own command (or dot-card); the periodic small-signal analyses (`.pac`, `.pnoise`, `.pxf`, `.psp`) run as dot-cards around a `pss/.pss` operating point.

`sp` is the small-signal S-parameter analysis — tag the ports with `portnum/z0` sources:

```

* LC low-pass S-parameters (50 ohm ports)
L1 in out 1u
C1 out 0 400p
V1 in 0 dc 0 ac 1 portnum 1 z0 50
V2 out 0 dc 0 ac 1 portnum 2 z0 50
.sp dec 50 1meg 100meg
.control
run
plot vdb(S_2_1)
.endc
.end

```



Command	What it does
<code>sp</code>	S-parameters + noise figure (<code>.sp</code>), Touchstone-exportable
<code>pss</code>	periodic steady state (shooting); the base for the periodic small-signal set
<code>hb</code>	harmonic-balance steady-state spectrum: <code>hb <f0> <K></code>
<code>hbosc</code>	autonomous HB for an oscillator: <code>hbosc <oscnod> <K> [fguess] [tstab]</code>
<code>phasenoise</code>	oscillator phase-noise $L(\Delta f)$ around an <code>hbosc</code> point
<code>qpss</code>	two-tone quasi-periodic steady state: <code>qpss <expr> <f1> <f2> ...</code>
<code>qpac /</code>	two-tone periodic AC / transfer / noise around a <code>qpss...hb</code> point
<code>qpxf /</code>	
<code>qpnoise</code>	
<code>envelope</code>	envelope following: <code>envelope <node> <fc> <tstop></code>
<code>stb</code>	loop-gain stability (Middlebrook/Tian double injection); reports phase/gain margin
<code>loadpull</code>	PA load-pull (and source-pull, <code>-source</code>): sweep the load impedance Γ over the Smith chart, per-point large-signal <code>.tran</code> , contour <code>Pout / gain / PAE / efficiency</code> ; view with <code>pyplot -contour gamma_re gamma_im pout_dbm</code>
<code>rfstab</code>	two-port RF stability from S-parameters: Rollett K and

Periodic small-signal — `.pss`, `.pac`, `.pnoise`, `.pfx`, `.psp`

These analyses characterise a circuit around a large-signal periodic operating point (a mixer under its LO, a switched-capacitor under its clock, an oscillator). `.pss` finds that periodic steady state; the rest are small-signal analyses *on top of it*. All share the same leading arguments `<fund> <tstab> <outnode> <points> <harmonics> ....`

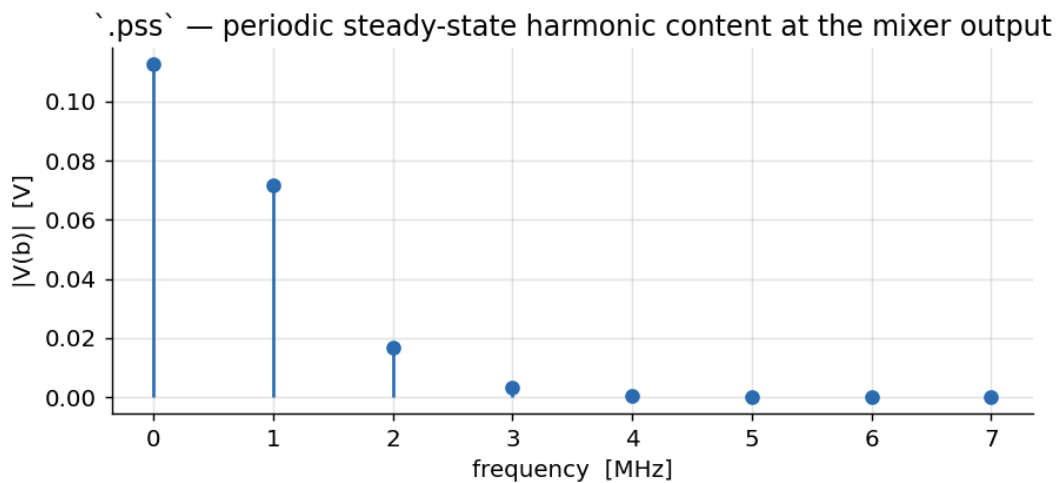
A self-contained example — a behavioural (B-source) conductance pumped by a 1 MHz “LO”, which acts as a mixer:

```

* LO-pumped behavioural mixer
V1 lo 0 SIN(0 1 1meg)
VDC a 0 DC 1 AC 1
R1 a b 10k
B1 b 0 I=(1m + 0.8m*v(lo))*v(b)      ; conductance modulated by the LO
C1 b 0 100p
.pss 1meg 1u b 1024 8 50 5u          ; find the periodic steady state
.control
run
setplot pss2                          ; frequency-domain result
plot mag(b)
.endc
.end

```

.pss returns the periodic operating point — its harmonic content shows the LO and the harmonics the nonlinearity generates:

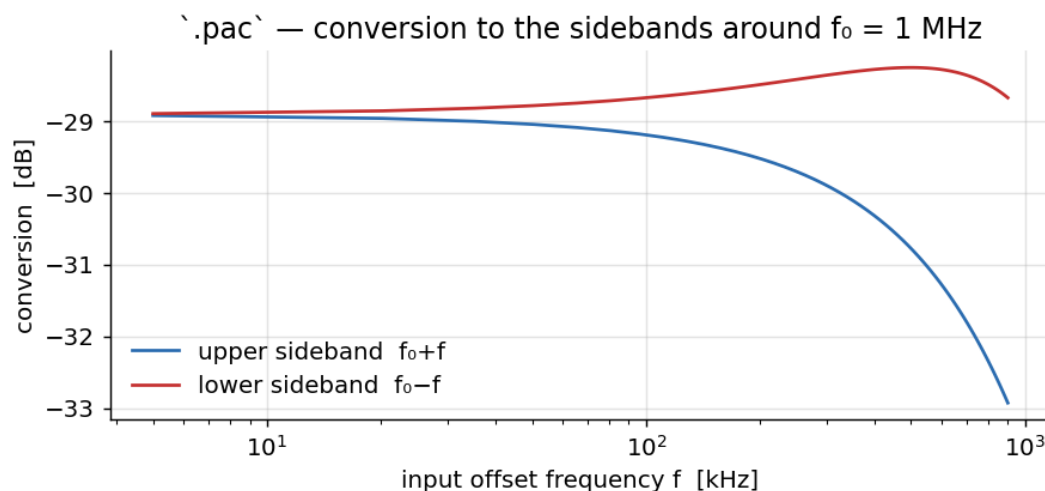


With that operating point, .pac computes the small-signal conversion to each mixing sideband ($f_0 \pm f$) versus the input frequency — swap the .pss line for:

```

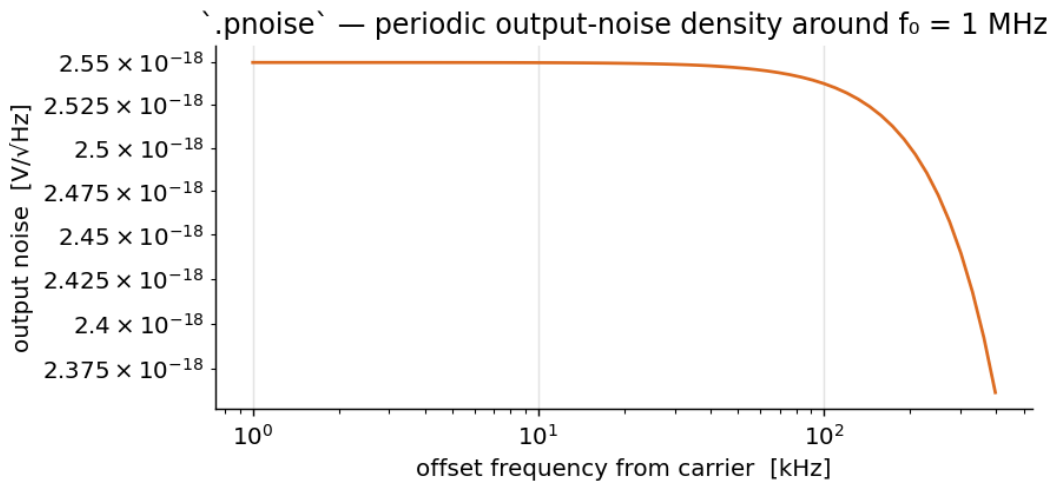
.pac 1meg 1u b 1024 6 50 5u lin 60 5k 900k 1
plot db(b_usb1) db(b_lsb1)      ; upper / lower sideband conversion

```



.pnoise gives the periodic / phase-noise density folded from every sideband, versus offset from the carrier:

```
.pnoise 1meg 1u b 1024 6 50 5u b vdc dec 25 1k 400k
set sqrnoise
plot onoise_spectrum          ; V/√Hz vs offset frequency
```



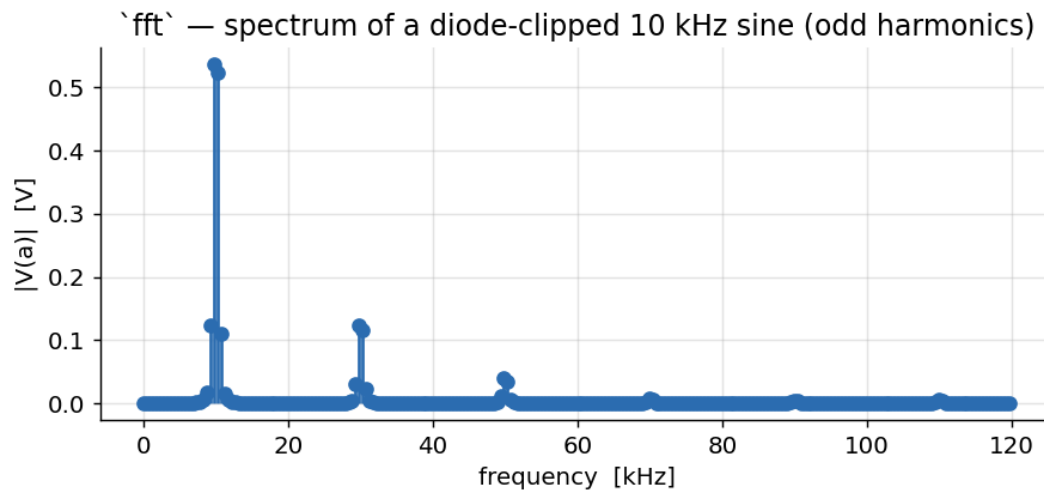
.pxf (periodic transfer function — sensitivity of an output to a small perturbation at each sideband) and .psp (periodic S-parameters) take the same .pss preamble; see the [RF suite reference](#) for those plus the two-tone qpss/qpac/qpnoise/qpxf family and the oscillator hbosc/phasenoise flow.

4. Signal processing and measurement

These operate on vectors already in memory (typically after a .tran).

fft <vec> computes an FFT (run linearize first so the samples are uniformly spaced); spec and psd are related spectral views.

```
* harmonics of a diode-clipped 10 kHz sine
Vin in 0 sin(0 1.2 10k)
R1 in a 1k
D1 a 0 DL
D2 0 a DL
.model DL D(is=1e-12)
.tran 1u 2m
.control
run
linearize
fft v(a)
plot mag(v(a))
.endc
.end
```



The symmetric clipper produces only odd harmonics (30, 50, 70 kHz ...).

Command	Purpose
meas	measure a quantity from a result (meas tran tr TRIG ... TARG ..., meas ac gain MAX vdb(out), gain/phase margin, delays, RMS, ...)
fft	FFT spectrum of a vector
spec	spectrum over a frequency window: spec <fstart> <fstop> <fstep> <vec>
psd	power spectral density via FFT
fourier	Fourier coefficients + THD of a transient
eye	eye diagram + jitter: eye <expr> -ui <T> [-tstart t0] [-threshold v]
linearize	resample a transient onto a uniform time grid (needed before fft)
cutout	keep only a slice of a vector
transpose	transpose a multi-dimensional vector
cross	build a vector by picking elements from others
compose	build a vector from values/ranges: compose v start=0 stop=1 step=0.1
reshape	change a vector's dimensions
settype / deftype	set/redefine a vector's type (voltage, time, ...) for axis labelling

5. Vectors, variables and expressions

Results are vectors; `let` creates or modifies them and `print/plot` read them. Expressions support the usual math plus `v()`, `i()`, `vdb()`, `vp()`, `mag()`, `ph()`, `real()`, `imag()`, `db()`, `abs()`, `sqrt()`, `vector()`, `length()`, `mean()`, `vecmax()`, `vecmin()`, and the ternary `?:`.

```
.control
run
let pwr = v(out)*i(vout)           ; a new vector
let gain_db = vdb(out)
print vecmax(abs(v(out)))          ; scalars are 1-length vectors
print mean(pwr)
.endc
```

Command	Purpose
let	assign a vector: let x = expr

Command	Purpose
unlet	delete vectors
print	print vector values: <code>print [col] expr ...</code>
display	list the vectors in the current plot
define / undefine	user function: <code>define sq(x) x*x</code>
set / unset	set/clear a control variable: <code>set numdgt=8</code>
setcs	like set, but keeps the value's case
option / options	set a simulator option: <code>option reltol=1e-4 klu</code>
setscale	change the plot's default scale vector
setseed	reseed the RNG for reproducible Monte Carlo
remzerovec	drop zero-length vectors

6. Plotting and data export

```
.control
run
plot v(out) vs time
plot vdb(out) xlog           ; log-x Bode
pyplot acmag vdb(out)       ; matplotlib PNG/-window
wrdata out.dat v(out) v(in)  ; columnar (whitespace) data file
wrdata -csv out.csv v(out) v(in) ; comma-separated, with a header row
write out.raw v(out)         ; binary rawfile (reload with `load`)
.endc
```

Command	Purpose
plot	the built-in plotter: <code>plot expr ... [vs expr] [xl lo hi] [yl lo hi] [xlog] [ylog]</code>
pyplot	render/export via matplotlib (<code>-eye</code> , <code>-hist</code> , <code>-contour</code> variants); PNG when a file is named; a hardcopy's exit status is published as <code>pyplot_status</code> and a failure names the image not written (E-547); the data table is <code><name>.npy</code> , a structured numpy array, or <code>.data</code> text under <code>set pyplot_export=ascii</code> , and <code>pyplot -export <sigs></code> writes just that (E-549)
gnuplot	render/export via gnuplot
hardcopy	write a PostScript/SVG hardcopy of a plot
asciiplot	plot as ASCII art in the terminal
wrdata	write columnar (whitespace) data — each vector prefixed by its scale; <code>-csv</code> (or <code>set wr_csv</code>) writes a comma-separated file with a header row and one shared scale column
write	write a binary rawfile (load reads it back)
load	load a rawfile into a new plot
wrsnp / rdsnp / wrs2p	Touchstone <code>.sNp</code> write / read / 2-port write
diff	compare two plots vector-by-vector
setplot	switch the current working plot
destroy	free a plot's data

7. Circuit and device management

```
.control
source amp.cir          ; read a deck
listing                 ; show the parsed circuit
alter r1 = 2k           ; change a device value, then re-run
altermod nmos vth0=0.4  ; change a .model parameter
alterparam vsup=3.3     ; change a .param, then `reset` to apply
show m1                 ; print device operating-point parameters
run
.endc
```

Command	Purpose
source	read and load a netlist file
listing	print the current circuit (logical/physical/deck)
edit	edit the deck in \$EDITOR, then reload
cirbyline	feed the circuit one line at a time (scripting)
alter / altermod / alterparam	change device / .model / .param values
show / showmod	print device / model parameters (operating point)
devhelp	list device types and their parameters
inventory	count the devices in the circuit
save	select which node/branch outputs to keep
setcirc	switch between loaded circuits
remcirc / removecirc	drop the current circuit
reset	rebuild the circuit (apply alterparam, clear state)
dump	dump the circuit matrix structure

8. Compiling and loading models

This build compiles Verilog-A to OSDI with `openvaf-r` and loads it directly.

```
.control
pre_osdi bsim4.osdi      ; load a compiled Verilog-A model (the `osdi` command)
pre_osdi -f bsim4.osdi   ; force-reload after recompiling (no restart)
pre_snp filter.s2p       ; compile a Touchstone file to an OSDI n-port, then load it
codemodel extra.cm       ; load an XSPICE code-model library
.endc
N1 d g s b bsim4mod      ; an OSDI device instance (N-prefix)
.model bsim4mod bsim4va w=10u l=1u
```

Command	Purpose
osdi (pre_ osdi)	load one/more compiled .osdi models; -f force-reloads a recompiled file — the deck whose control block carries it is parsed afterwards and is built against the new object, while a circuit loaded earlier is named at the reload and its next run refused until reset / re-source rebuilds it (2026-09-05 hunt, F16: it would otherwise run the new object's code on the old layout's data)
snp (pre_snp)	compile a Touchstone .snp to a Verilog-A n-port OSDI model, then load it
codemodel	load an XSPICE code-model (.cm) library
use	load a device library

9. Statistics, yield and optimization

```
* Monte-Carlo yield of an RC corner
.param rr=agauss(1k, 5p, 3)          ; 5% (3-sigma) Gaussian on R
R1 in out {rr}
C1 out 0 1n
Vin in 0 ac 1
.ac dec 20 1 1meg
.control
montecarlo 500 -analysis ac -spec 'vdb(out)' -min -3.5 -max -2.5
.endc
.end
```

Command	Purpose
<i>strings</i>	r"..." keeps its case and spaces through the deck reader; f"... {expr:.3f} ..." is evaluated when the command runs; rf"..." both (E-553)
montecarlo	packaged yield: N samples, per-spec pass/fail, yield + 95% CI; -expr [name=]<expression> records a value per sample, unjudged, into a montecarlo<n> plot (a scalar as an N-long vector, a waveform as an $N \times L$ family) with no yield unless a limited -spec is given (E-552); -track "<track arguments>" runs track after every sample's analysis and records its hits into a plot of its own, track<k> (\$track_plot): sample as its scale, hits per sample, and each track-plot vector under its own name (time/frequency/v_sweep, value, index, x_out, width) as an $L_{\max} \times N$ family whose row k is hit k of every sample, nan where a sample had fewer (E-582, E-584)
mcsample	Latin-Hypercube sampling of the .param random draws (lower variance)
highsigma	rare-event / high-sigma failure probability by scaled-sigma importance sampling
mccorr	register a correlation matrix; draw with mvnorm() in .param
setseed	reproducible random draws
sweep	sweep any knob (device/model/.param) and record outputs into a plot
wcd	worst-case distance / most-probable failure point: the shortest distance in standardised parameter space from nominal to a spec boundary, i.e. the sigma level at which the design first fails (Enhancement-305)
corners	run the analysis at every process corner the loaded Verilog-A models declare ((* corner="..." *) on a parameter, Enhancement-654), the nominal tt first, recording each -output into a corners<n> plot on a corner index scale; -mc N <montecarlo arguments> runs a montecarlo per corner instead and records yield, npass, nsamples and nfailed per corner
writemc	put each value (a scalar, evaluated on the current plot) onto the row .option savemc wrote for the last analysis run — a value computed in the .control block, beside the draws behind it; a column is added on first use (writemc [name=]<expression> ...)
writecorner	the same as writemc, under the corner file's name: each value onto the row .option / savecorner (and .option savemc, when set) wrote for the last analysis run; on an
writecr	autocorner combined plot each value goes onto every corner's own row (Enhancement-742)
optimize	Nelder-Mead / Levenberg-Marquardt parameter optimizer against an objective

See the [statistics](#) and [optimizer](#) references for full treatments with plots.

10. Post-layout and reliability

Command	Purpose
reduce	reduce an R/C parasitic network (TICER) to an equivalent .subckt valid to fmax
aging	age aging-capable devices to a target lifetime (HCI/NBTI/TDDb) for fresh-vs-aged runs
emir	power-grid IR-drop + electromigration current density + Black's-equation MTTF

11. Interactive debugging and control

```
.control
stop when v(out) > 0.9      ; breakpoint on a condition
trace v(out)                ; watch a node
run                          ; runs until the breakpoint
where                        ; last non-converging node/device
resume                      ; continue
savestate ckpt.st           ; checkpoint the transient state to disk
* ... later, even in a fresh process ...
loadstate ckpt.st           ; restore and continue the .tran
.endc
```

Command	Purpose
run	run the analysis from the deck's dot-cards (optionally to a rawfile)
resume	continue after a breakpoint/stop
stop	set a breakpoint (stop after N, stop when expr)
step	advance N timepoints
trace / iplot	watch / incrementally plot nodes during a run
status / delete	list / remove breakpoints and traces
savestate / loadstate	checkpoint / restore the full transient state (cross-process)
snsave / snload	save / load a solution snapshot
where	print the last non-converging node or device

12. Control-flow scripting (.control)

.control blocks are a small shell-like language. In addition to the commands above:

```
.control
run
* structured control
if v(out) > 0.5
  echo "high"
else
  echo "low"
end
* loops
foreach f 1k 10k 100k
  ac dec 10 $f $f
  print vdb(out)
```

```

end
let k = 0
dowhile k < 3
  echo iteration $k
  let k = k + 1
end
.endc

```

Construct	Purpose
if / else / end	conditional
while / dowhile / end	pre-/post-test loops
foreach var v1 v2 ... / end	iterate over a value list
repeat [N] / end	repeat N times (or forever)
break / continue	loop control
label / goto	jump within a block
shift	shift a list/argv variable left
echo	print text/\$variables
shell	run a host shell command
cd / getcwd	change / print working directory
strcmp / strstr / strslice	string compare / search / slice into \$var
fopen / fread / fclose	open / read / close a file into shell variables

13. Help and utility commands

Command	Purpose
help / tutorial / newhelp / oldhelp	documentation browser (help <command>)
history	recall previous commands (history [-r] [N])
alias / unalias	define / remove a command alias
synhl	preview interactive syntax highlighting of a line
version	print the version
sysinfo	print host CPU/memory summary
rusage	print resource usage (time, memory, all)
cdump	dump the parsed control structures
mdump / mrdump	dump the matrix / RHS (debugging)
bug	open a bug report
rehash	rebuild the host-command hash
quit / exit	exit ngspice (exit is the same as quit)

Developer/edge commands: aspice/rspice/jobs (async/remote runs), mc_source, optran, wrnodev, check_ifparm, bltplot (a plot alias), state (unimplemented placeholder). use exists in the command table but carries no help text, so help all does not list it.

14. Every command **help all** lists

165 commands, alphabetically, as the build's help all prints them (name, arguments, one-line description; regenerated by make_commands_table.py, checked by the helpcmd suite).

Command	Arguments	Description
ac	[.ac line args]	Do an ac analysis.
aging	t_target [rate opvar] [param ageparam] [dynamic tstop [tstep]] [verbose]	age every aging-capable device to a target lifetime (stress -> degrade -> re-stamp) for fresh-vs-aged reliability simulation.
alias	[[word] alias]	Define an alias.
alter	dev param=value [param=value ...] @dev[param]=value dev=value	Alter device parameters (every pair applies; Enhancement-586).
altermod	model param=value [param=value .. .] @model[param]=value	Alter model parameters (every pair applies; a bare word reaches a string parameter; Enhancement-586).
alterparamdevspecs	parmname value :	Alter .param parameters.
asciiplot	plotargs	Produce ascii plots.
aspice	file [outfile]	Run a spice job asynchronously.
break		Break out of a block.
bug		Report a ngspice bug.
cd	[directory]	Change working directory.
cdump		Dump the current control structures.
check_ifparm		Check model ifparm descriptors (for developers).
cirbylineline		Enter a circuit line.
codemodel	library library ...	Loads the code model libraries.
compose	var parm=val ...	Compose a vector.
continue		Continue a loop.
corners	[-list <c1>[,<c2>...]] [-nonominal] [-analysis <cmd>] [-output <expr> ...] [-mc <N> <montecarlo arguments>]	run the analysis at every process corner the loaded Verilog-A models declare ((* corner="..."*) on a parameter, Enhancement-654), the nominal tt first, and record each -output into a corners plot on a corner index scale (\$corners_plot, \$corners_names, \$corners_n); with -mc, a montecarlo N <arguments> per corner instead, recording yield, npass, nsamples and nfailed per corner. The corner variable is put back afterwards; a .option savemc file tags each row with its corner.
cross	vecname number [vector ...]	Make a vector in a strange way.
cutout	[vec ...]	Cut out portion of a vector.
dc	[.dc line args]	Do a dc analysis.
define	[[func (args)] stuff]	Define a user-definable function.
deftype	spec name pat ...	Redefine vector and plot types.
delete	[all] [break number ...]	Delete breakpoints and traces.
destroy	[plotname] ...	Throw away all the data in the plot.
devhelp	devspecs	Show device information.
diff	plotname plotname [vec ...]	'diff' two plots.
display		Display vector status.
disto	[.disto line args]	Do a distortion analysis.
dowhile	condition	Execute while the condition is TRUE.
dump		Print a dump of the current circuit.
echo	[stuff ...]	Print stuff.
edisplay	node node ...	Print all event nodes.
edit	[filename]	Edit a spice deck and then load it in.
else		Goes with if.

Command	Arguments	Description
emir	[rail V] [thresh frac] [thick m] [jmax A/m2] [n exp] [tref s] [top k] [verbose]	power-grid reliability -- worst IR-drop and per- segment electromigration current density + Black's-equation MTTF.
end		End a block.
envelope	node fc tstop [nppp N] [m M0] [maxm Mmax] [reltol t] [settle ts]	envelope following -- the slow amplitude/phase envelope of a carrier-driven circuit (implicit monodromy period jumps).
eprint	node node ...	Print event values.
eprvcd	[-a] [-t timescale] node node ...	Print event values into VCD file.
esave	all none node node ...	Save event values.
exit		Quit ngspice (the same as quit).
eye	<expr> -ui <T> [-tstart t0] [- threshold v]	eye diagram + jitter (SerDes).
fclose	handle	Close an open file.
fft	vector ...	Create a frequency domain plot with FFT.
fopen	handle file_name [mode]	Open file_name with mode, return handle in \$handle.
foreach	variable value ...	Do once for each value.
fourier	fund_freq vector ...	Do a fourier analysis of some data.
fread	handle result [length]	Read a line from open file handle, data in \$result, status in \$length.
getcwd	[directory]	Print current working directory.
gnuplot	file plotargs	Send plot to gnuplot.
goto	word	Go to a label.
hardcopy	file plotargs	Produce hardcopy plots.
hb	f0 K [points] [maxiter]	harmonic-balance steady-state spectrum (K harmonics).
hbosc	oscnode K [fguess] [tstab]	autonomous harmonic balance -- an oscillator's steady state (harmonics + oscillation frequency), seeded from a transient.
help	[subject] ...	Hierarchical documentation browser.
highsigma	<N> [-scale <lambda>] [-inflate <param>]... [-seed <s>] [- analysis <cmd>] -metric <expr> [- max <hi>] [-min <lo>]	rare-event (high-sigma) failure-probability estimation by scaled-sigma importance sampling -- inflates Gaussian .param sigmas by lambda so tail failures (metric > hi or < lo) are sampled often, reweights, and reports P(fail), relative error, and equivalent sigma-to-fail. -inflate (E-538) names which statistical parameters may be inflated (vth0, or @mod [vth0]); without it every one is, and the importance weight -- a product over the inflated dimensions -- collapses on any deck with per-instance mismatch. The reported effective sample size says when that has happened.
history	[-r] [number]	Print command history.
if	condition	Execute if the condition is TRUE.
inventory		Print circuit inventory.
iplot	[-w width] [-d initial_steps] [- o] [all] [node ...]	Incrementally plot nodes.
jobs		Report on asynchronous spice jobs.
label	word	Create someplace to go to.

Command	Arguments	Description
let	varname = expr	Assign vector variables.
linearize	[vec ...]	Convert plot into one with linear scale.
listing	[logical] [physical] [deck] [expand] [tree]	Print the current circuit.
load	file ...	Load in data.
loadpull	-load <R> <L> <C> -out <node> - drive <Vsrc> -f <freq> [-supply <Vsrc>] [-z0 50] [-n 15] [-gmax 0. 85] [-nper 20] [-source]	load-pull (or source-pull) analysis for PAs. Sweeps the load impedance Gamma over the Smith chart, runs a large-signal .tran at each point, and stores gamma_re,gamma_im,pout_dbm,gain_db[,pae,eff] in a 'loadpull' plot (contour with pyplot -contour).
loadstate	file	Restore a transient checkpoint and continue the .tran run.
mc_		Re-source the actual circuit deck for MC simulation.
source		
mccorr	<k> <m11> <m12> ... <mkk> off	register a k x k correlation matrix (row-major) for process/mismatch sampling; then use mvnrm(1..k) in .param expressions to draw correlated standard normals (one shared draw per Monte Carlo sample).
mcsample	lhs <N> [seed <s>] random off	Latin-Hypercube Monte Carlo sampling -- the .param agauss/gauss/aunif/unif/limit draws become stratified so N reset-driven runs cover each random dimension evenly (lower-variance estimates than plain MC).
mdump	[outfile]	Dump the current matrix.
meas	various ...	User defined signal evaluation.
montecarlo	<N> [-lhs] [-warm] [-seed <s>] [- analysis <cmd>] (-spec <metric> - max <hi> -min <lo>)... (-expr [name=]<expression>)... (-track "<track arguments>")... [-writemc [name=]<expression> ...]	packaged Monte Carlo -- runs N samples. A -spec with a limit is judged: a sample passes only if every spec is within its limits, and the yield is reported with a Wilson 95% CI and per-spec violations. An -expr is recorded per sample, unjudged, into a montecarlo plot (<i>montecarlo_plot</i>) : <i>ascalarasanN - longvectoronthe`sample`scale,awaveform(adc/acsv trackruns`track < arguments >` afterevery sample`sanalysisandrecordsitshitsintoa, k > (track_plot): sample as its scale, hits per sample, and every vector of the track plot under its own name (time/frequency/v_sweep, value, index, x_out, width) as an Lmax x N family whose row k is hit k of every sample, nan where a sample had fewer (plain vectors when no sample has more than one). Any of the three (-lhs for a lower-variance estimate; correlations via mvnrm(), corners via .lib). With .option savemc, -writemc puts each listed value (an -expr name, a track., any scalar expression) onto the sample's row of the savemc file, as the writemc command does after a run.</i>
mrddump	[outfile]	Dump the current RHS to file.

Command	Arguments	Description
newhelp	[command name] ...	help.
noise	[.noise line args]	Do a noise analysis.
oldhelp	[command name] ...	Print help.
op	[.op line args]	Determine the operating point of the circuit.
optimize	(-param -mparam -dparam) name init lo hi ... -analysis <cmd> (- minimize <expr> -target <expr> <val> [<w>] ...) [-method nm lm] [-maxiter N] [-tol T] [-verbose]	parameter optimizer (Nelder-Mead / least- squares Levenberg-Marquardt; -param = alter device/instance, -mparam = @model[param] via altermod, -dparam = symbolic .param via re- source; multi-analysis).
option	[option] [option = value] ...	Set a simulator option.
options	[option] [option = value] ...	Set a simulator option.
optran		Prepare optran by setting 6 flags.
osdi	[-f] [-va] library ...	Loads one or more OSDI libraries (use as pre_ osdi file.osdi). With -va, a .va argument is compiled by openvaf-r into an osdi/ directory beside the netlist and the object it produces is loaded; recompiled every run unless the deck carries .option osdicache. With -f, force- reloads an already-loaded file so a recompiled model is picked up without restarting.
phasenoisefstart fstop [points]		oscillator phase-noise spectrum L(df) via the PPV, around the hbosc operating point.
plot	expr ... [vs expr] [xl xlo xhi] [yl ylo yhi]	Plot things.
pre_ osdi	[-f] [-va] library ...	the same as osdi. In a deck's .control block the pre_ prefix runs the line before the circuit is read; at the prompt it loads (or with -f reloads) now.
print	[col] expr ...	Print vector values.
psd	vector ...	Create a power spectral density plot with FFT.
pss	[.pss line args]	Do a periodic steady state analysis.
pyplot	[file] plotargs -export <sigs> -hist <sigs> -contour <z> <x> <y> -smith <sigs> -fft <sigs> -bode -nyquist -polar <sigs> -eye <expr> -ui <T>	matplotlib plots (file name optional; the plot keywords xlog, xlim, vs, title ... apply). -export writes the data table only, .npz (set pyplot_ export=ascii for .data). Settings: pyplot_ terminal=png svg pdf, pyplot_python, pyplot_ backend, pyplot_style, pyplot_figsize, pyplot_ subplots, pyplot_grid, pyplot_legend, pyplot_dpi, pyplot_decimate=auto off, pyplot_eng=off ...; pyplot_status holds a hardcopy's exit status.
pz	[.pz line args]	Do a pole / zero analysis.
qpac	f_in <dec oct lin> N fstart fstop	two-tone QPAC -- sideband response (single f) or swept plot vs f_in, around the qpss ... hb operating point.
qpnoise	output_node f_in [cyclo] output_ node <dec oct lin> N fstart fstop	two-tone QPnoise -- output/input noise density at f_in (or swept onoise/inoise plot), around the qpss ... hb operating point (cyclo = cyclostationary PSD).
qpss	expr f1 f2 [periods] [maxorder] expr f1 f2 hb [K1] [K2]	two-tone quasi-periodic steady state (transient / frequency-domain HB, incommensurate- capable).

Command	Arguments	Description
qpxf	output_node f_in output_node <dec oct lin> N fstart fstop	two-tone QPXF -- per-sideband transfer at f_in (or swept xf/xf_conv plot), around the qpss ... hb operating point.
quit		Quit ngspice.
rdsnp	file [nports]	Read a Touchstone 1 or 2 file (S, Y or Z; RI, MA or DB; any port count) into a new plot with complex S _{i_j} vectors, Rbase and, for a v2 file, Zref.
reduce	fmax [factor f] [file fname] [name subckt] [keep node ...]	reduce the R/C parasitic network (TICER) to an equivalent .subckt preserving port behaviour over DC..fmax.
rehash		Rebuild the unix command database.
remcirc		Remove current circuit.
removecirc	[circuit name]	Remove the current circuit from memory.
remzerovec		Remove zero-length vectors.
repeat	[number]	Repeat number times, or forever.
reset		Terminate a simulation after a breakpoint (formerly 'end').
reshape	vector ... [shape]	change the dimensions of a vector.
resume		Continue after a stop.
rfstab	[S11 S12 S21 S22]	two-port stability / gain report from .sp S-parameters. Computes Rollett K, Delta , the mu/mu' stability factors and the max stable/available gain (MSG/MAG) per frequency, stores them (k, magdelta, mu, mu_src, gmax, msg, stable) in an 'rfstab' plot, and reports the unconditional-stability verdict. With no args uses the .sp defaults S_1_1 S_1_2 S_2_1 S_2_2.
rspice	[input file]	Run a spice job remotely.
run	[rawfile]	Run the simulation as specified in the input file.
rusage	[resource ...]	Print current resource usage.
save	[all] [node ...]	Save a spice output.
savestate	file	Save the current transient state to a checkpoint file.
sens	[.sens line args]	Do a sensitivity analysis.
set	[option] [option = value] ...	Set a variable.
setcirc	[circuit name]	Change the current circuit.
setcs	[option] [option = value] ...	Set a variable, case remains as given.
setplot	[plotname]	Change the current working plot.
setscale	[vecname [vecname]]	Change default scale of current working plot, or set/clear the scale for a single vector.
setseed	[seed value]	Reset the random number generator with new seed value.
settype	type vec ...	Change the type of a vector.
shell	[args]	Fork a shell, or execute the command.
shift	[var] [number]	Shift argv or the named list var to the left.
show	devspecs ...	parmspecs ... : Print out device parameters.
showmod	models ...	parameters ... : Print out model summary.
snload	netlist snapshot	Load a snapshot (needs BOTH the circuit file and the snapshot).

Command	Arguments	Description
snp	<code>[-osdi -native] [-maxerr <x> -force] [-maxpoles <N> -order <N>] file.sNp [module]</code>	compile a Touchstone S-parameter file to a Verilog-A n-port OSDI model (or, -native, to a .nport fit file for the built-in device). Use as <code>pre_snp file.sNp</code> (runs before circuit parsing); load the .osdi it writes with <code>pre_osdi file.osdi</code> . A fit whose rms relative error is above 0.1 is refused and nothing is written; -maxerr raises the limit, -force removes it. The order climb stops at 128 poles by default and says so when it gets there; -maxpoles sets the cap, -order pins the pole count.
snsave	file	Save a snapshot.
source	file	Source a ngspice file.
sp	<code>[.sp line args]</code>	Do an S-parameter analysis.
spec	<code>start_freq stop_freq step_freq vector ...</code>	Create a frequency domain plot.
state	(unimplemented)	Print the state of the circuit.
status		Print the current breakpoints and traces.
stb	<code><Vprobe> <Iprobe> (dec oct lin <N> <fstart> <fstop>)</code>	stability / loop-gain analysis (Middlebrook/Tian double injection). Vprobe = series 0 V probe in the loop (+node = driver A, -node = load B); Iprobe = shunt 0 A probe (ground -> load node B). Reports phase/gain margin and stores complex 'loopgain'.
step	[number]	Iterate number times, or one.
stop	[stop args]	Set a breakpoint.
strcmp	varname s1 s2	Set \$varname to strcmp(s1, s2).
strslice	varname s1 offset length	Set \$varname to s1[offset ... offset+length].
strstr	varname s1 s2	Set \$varname to strstr(s1, s2).
sweep	<code><knob> (<start> <stop> <step> lin dec oct <N> <start> <stop> list <v>...) [-analysis <cmd>] [-output <expr> ...]</code>	sweep any knob and record outputs into a plot (auto-detects device/instance/source via alter, @model [param] via altermod, .param via resource).
synhl	[command line ...]	Preview interactive syntax highlighting of a line.
sysinfo		Print out system info summary.
tf	<code><output> <input-source></code>	Do a small-signal DC transfer function analysis (gain, plus input and output impedance); is v(node), v(node1,node2), or i(vsource).
trace	[all] [node ...]	Trace a node.
track	<code><expr> [<expr> ...] [-range x0 x1] [-spec <spec>] [-analysis <plot type>] [-which all first last N -N] [-edge rise fall both] [-at entry exit mid] [-prominence p] [-raw] [-output name ...]</code>	every place a condition holds, as a plot trackN -- a locator (localmax, localmin, globalmax, globalmin, bare or on an expression), a crossing lhs==rhs, a region lhs<rhs, or any boolean; one spec, every expression read at the hits.
tran	<code>[.tran line args]</code>	Do a transient analysis.
transpose	varname ...	Perform matrix transposition on multi-D vectors.
tutorial	[subject] ...	Hierarchical documentation browser.
unalias	word ...	Undefine an alias.
undefine	[func ...]	Undefine a user-definable function.
unlet	varname ...	Undefine vectors.

Command	Arguments	Description
unset	varname ...	Unset a variable.
version	[number]	Print the version number.
wcd	-metric <expr> [-max <hi>] [-min <lo>] [-analysis <cmd>] [-maxiter <n>] [-tol <t>] [-step <h>] [-is <N>] [-seed <s>]]	worst-case distance / most-probable-failure-point high-sigma -- walks standardised normal space to the closest failure point, reports that distance beta and the first-order probability Phi(-beta), and can refine it with mean-shift importance sampling centred there.
where		Print last non-converging node or device.
while	condition	Execute while the condition is TRUE.
wrdata	file plotargs	Send plot data to file.
write	file expr ...	Write data to a file.
writecorner	[name=]<expression> ...	the same as writemc, under the corner file's name: put each value (a scalar, evaluated on the current plot) onto the row .option savecorner -- and .option savemc, when set -- wrote for the last analysis run; on an autocorner combined plot each value goes onto every corner's own row. One command serves both recorders today; the name is separate so the two can diverge.
writecr	[name=]<expression> ...	short for writecorner -- writemc under the corner file's name: each value onto the row .option savecorner (and .option savemc, when set) wrote for the last analysis run.
writemc	[name=]<expression> ...	put each value (a scalar, evaluated on the current plot) onto the row .option savemc wrote for the last analysis run -- a value computed in the . control block, beside the draws behind it; a column is added on first use.
wrcodev		Save current node voltage values to file.
wrs2p	file	Send s-param data to file.
wrsnp	file [ri ma db] [s y z] [hz khz mhz ghz]	Send N-port network data to a Touchstone file.

Generated for the ngspice-46 + OpenVAF-r enhancement project. Plots produced by make_commands_figs.py from real ngspice runs, the table of section 14 by make_commands_table.py from the build's own help all; rebuild the PDF with build_commands_pdf.py.